

AI MEDIA ECOSYSTEM — BLUEPRINT PACK

Dokumen ini adalah paket awal repo-ready untuk membangun sistem:

auto schedule

auto post

humanic writing

auto daily publishing

MCP + API orchestration

multi-platform repurpose

analytics feedback loop

monetization-ready publishing engine

Target platform list:

Medium

Blogger

WordPress

Substack

Beehiiv

Mirror.xyz

Dev.to

Hashnode

Ghost.org

Catatan: beberapa platform punya API resmi yang kuat, beberapa terbatas, dan beberapa butuh kombinasi API + webhook + browser automation. Blueprint ini dibuat supaya bisa jalan walau level integrasi tiap platform berbeda.

1) REPO STRUCTURE

```
ai-media-ecosystem/ ├── README.md ├── ARCHITECTURE.md ├── DESIGN.md ├── PRD.md
├── STRUCTURE.md ├── AGENTS.md ├── MCP.md ├── API.md ├── AUTOMATION.md ├──
SCHEDULER.md ├── CONTENT_ENGINE.md ├── REPURPOSE.md ├── SEO.md ├──
MONETIZATION.md ├── ANALYTICS.md ├── MEMORY.md ├── LOGS.md ├──
CHANGELOG.md ├── PROMPTS.md ├── PROMPT_MASTER_FINAL.md ├── SECURITY.md
├── DEPLOYMENT.md ├── FAILSAFE.md ├── OBSERVABILITY.md ├── ROADMAP.md ├──
STYLE_GUIDE.md ├── DATABASE.md ├── VECTOR_MEMORY.md ├── COSTS.md ├──
EXAMPLES.md ├── /agents | ├── writer-agent.md | ├── editor-agent.md | ├── seo-agent.md
| ├── repurpose-agent.md | ├── scheduler-agent.md | ├── publisher-agent.md | ├──
analytics-agent.md | ├── trend-agent.md | ├── monetization-agent.md | └──
memory-agent.md ├── /workflows | ├── n8n/ | ├── mcp/ | ├── api/ | └── cron/ ├── /templates
├── /content | ├── drafts/ | ├── scheduled/ | ├── published/ | ├── archived/ | └── assets/ ├──
/logs ├── /memory ├── /data ├── /scripts ├── /config └── /docs
```

2) README.md

Project Name

AI Media Ecosystem

Purpose

Membangun mesin publishing AI yang bisa:

menerima satu ide

membuat artikel humanic

memecah jadi banyak format

menjadwalkan otomatis

memposting ke banyak platform

membaca analytics

belajar dan mengoptimalkan performa

Core Outcome

Satu input ide → banyak output konten → multi-platform distribution → monetization loop.

Core Principles

humanic, bukan spam AI

markdown-first

automation-first

modular

API-native

MCP-ready

analytics-driven

fail-safe by design

Primary Platforms

WordPress as main hub

Medium for reach

Blogger for lightweight SEO + Google ecosystem

Substack / Beehiiv for newsletter and paid audience

Dev.to / Hashnode for tech authority

Ghost for premium publishing

Mirror.xyz for web3-friendly publishing

Main Flows

1. idea capture

2. research
3. draft generation
4. humanic editing
5. SEO optimization
6. repurposing
7. scheduling
8. publishing
9. analytics ingestion
10. optimization loop

3) PRD.md

Problem Statement

Publishing manual itu lambat, tidak konsisten, susah scale, dan rawan burnout. Dibutuhkan sistem yang mengubah satu ide menjadi banyak format yang bisa dipublish otomatis dengan kualitas natural.

Product Goal

Membangun AI publishing system yang mampu menjalankan content pipeline harian secara otomatis dan tetap terasa natural.

User Persona

solo creator

technical writer

AI builder

indie operator

newsletter operator

media entrepreneur

Success Metrics

jumlah post harian

publish consistency

CTR

time-to-publish

audience growth

conversion to email subscribers

affiliate/sponsor revenue

Functional Requirements

draft generation

humanic rewrite

auto schedule

auto post

multi-platform formatting

tag generation

metadata generation

image/thumbnail prompt generation

analytics capture

retry on failure

audit logs

Non-Functional Requirements

modular

secure

low-cost

observable

resumable

rate-limit aware

platform-safe

4) STRUCTURE.md

System Layers

Layer 1: Source of Truth

Tempat ide utama disimpan. Contoh:

Obsidian

Notion

Markdown repo

Google Docs

Layer 2: AI Core

Model routing dan prompt execution. Contoh:

OpenRouter

local model

hosted LLM

Layer 3: Workflow Engine

Orkestrasi otomatis. Contoh:

n8n

custom scripts

cron jobs

MCP server

Layer 4: Content Engine

drafting

editing

formatting

repurposing

SEO

Layer 5: Publisher Layer

WordPress

Medium

Blogger

Substack

Beehiiv

Dev.to

Hashnode

Ghost

Mirror.xyz

Layer 6: Analytics Layer

Google Analytics

Search Console

platform metrics

custom event tracking

Layer 7: Memory Layer

memory.md

logs

vector DB

performance history

5) ARCHITECTURE.md

Reference Architecture

[IDEA CAPTURE] ↓ [RESEARCH + CONTEXT] ↓ [LLM DRAFT] ↓ [HUMANIC EDIT] ↓
[SEO + METADATA] ↓ [REPURPOSE ENGINE] ↓ [SCHEDULER] ↓ [PUBLISHER]

ADAPTERS] ↓ [ANALYTICS INGESTION] ↓ [MEMORY UPDATE] ↓ [OPTIMIZATION LOOP]

Core Components

1. Content Intake

Inputs:

raw ideas

notes

source links

headlines

voice memos

trend signals

2. AI Orchestrator

Responsibilities:

select model

choose prompt template

route task to agents

enforce structure

return final payload

3. Humanic Writer

Responsibilities:

make text natural

remove robotic patterns

adapt tone

preserve clarity

keep personality consistent

4. Repurpose Engine

Converts one master article into:

long-form blog

newsletter

short post

thread

LinkedIn article

dev post

teaser copy

title variants

5. Scheduler

Responsibilities:

content queue

publish time selection

timezone handling

frequency control

daily cadence

6. Publisher Adapters

Adapters per platform. Each adapter handles:

auth

formatting

media upload

post creation

error handling

rate limits

7. Analytics Loop

Collects:

views

clicks

saves

open rates

CTR

comments

conversion

Then feeds memory to improve future output.

6) DESIGN.md

Design Objectives

simple to operate

easy to expand

low friction

audit-friendly

safe for scheduled automation

Content Style Design

Humanic Rules

short sentences when needed

vary rhythm

avoid repetitive AI phrases

use direct, grounded language

preserve user voice

keep substance over polish

UI/UX Principles (if dashboard is built)

content queue visible at a glance

status labels clear

scheduled/published/failed states visible

one-click retry

one-click edit before publish

analytics summary on top

Data Design Principles

one canonical article object

multiple derived variants

platform-specific payloads

versioning for all drafts

logs attached to every publish attempt

7) AGENTS.md

Agent Map

writer-agent

Creates the master draft. Input: idea, sources, target angle. Output: article draft.

editor-agent

Makes the draft humanic, clear, and natural.

seo-agent

Generates:

title

slug

meta description

keywords

headings

internal linking suggestions

repurpose-agent

Turns master article into platform-specific variants.

scheduler-agent

Selects publish time and builds queue.

publisher-agent

Sends content to each platform adapter.

analytics-agent

Reads performance data and scores content.

trend-agent

Detects what topics are rising.

monetization-agent

Optimizes for:

affiliate

newsletter signups

product mentions

sponsor readiness

memory-agent

Stores:

what worked

what failed

preferred tone

platform behavior

best posting cadence

8) MCP.md

MCP Role

MCP acts as the tool bridge between the AI and external systems.

MCP Targets

file system

content repo

database

CMS APIs

analytics sources

scheduling tools

browser automation tools

MCP Use Cases

read draft from vault

create platform payload

verify publish status

pull analytics

update memory

MCP Strategy

Use MCP for standardized tool access, while API adapters handle platform-specific publishing.

9) API.md

Platform Adapter Matrix

WordPress

strong REST API

best for central publishing hub

Blogger

usable API

good for Google ecosystem

Medium

limited direct automation; may require workaround layer depending on access pattern

Substack

limited native automation; best treated as newsletter channel with controlled integration

Beehiiv

good for newsletter automation and monetization

Mirror.xyz

treat as special-case publishing channel

Dev.to

API-friendly

good for tech distribution

Hashnode

API-friendly

good for developer audience

Ghost

strong admin API

good for premium publishing

Adapter Rules

Every adapter should support:

create draft

publish

schedule if supported

update existing post

attach tags/categories

log result

10) AUTOMATION.md

Main Automation Pipeline

Idea In ↓ Research ↓ Draft Generation ↓ Humanic Rewrite ↓ SEO Pack ↓ Repurpose Variants ↓
Schedule Queue ↓ Publish per Adapter ↓ Collect Metrics ↓ Update Memory

Automation Modes

Manual Draft

User approves before publish.

Semi-Auto

AI drafts, human approves, then auto-publish.

Full Auto

AI drafts and publishes within rules.

Daily Auto Mode

A fixed daily cycle that produces and posts one core piece plus derived posts.

Safety Rules

do not publish if quality score too low

do not publish if source confidence low

do not exceed rate limits

do not overwrite published content without versioning

11) SCHEDULER.md

Scheduler Functions

content queue management

timezone-aware posting

platform cadence control

delayed publish

retry queue

failover queue

Suggested Cadence

1 master article per day

1 newsletter version per day or per week

1 to 3 social repurposes per day

platform-specific cadence based on audience response

Scheduling Logic

Inputs:

platform

content type

audience timezone

engagement history

day of week

urgency score

Output:

optimal publish timestamp

12) CONTENT_ENGINE.md

Canonical Content Object

Fields:

id

title

subtitle

angle

source notes

body markdown

summary

tags

slug

platform variants

publish status

timestamps

metrics

Content Types

essay

article

tutorial

analysis

newsletter

thread

announcement

update

opinion

Humanic Content Rules

use concrete examples

avoid overexplaining obvious things

keep the voice grounded

prioritize clarity and rhythm

vary sentence length

13) REPURPOSE.md

Repurpose Outputs

A single article can become:

WordPress post

Medium story

Blogger article

Substack newsletter

Beehiiv newsletter draft

Dev.to technical post

Hashnode article

Ghost publication

LinkedIn article

X thread

short post teaser

Repurpose Rules

preserve meaning

adapt length

change formatting per platform

generate platform-native hooks

do not copy-paste blindly

14) SEO.md

SEO Tasks

keyword extraction

title variation

slug generation

meta description

heading structure

schema suggestions

internal linking suggestions

SEO Focus

search intent alignment

readability

topical authority

semantic depth

freshness

15) MONETIZATION.md

Monetization Paths

ads

affiliate

sponsored content

newsletter paid tier

consulting

digital products

content bundles

course funnels

Monetization Rule

Do not force monetization on every post. Build trust first.

Best Monetization Platforms

WordPress for flexible monetization

Beehiiv for newsletter monetization

Substack for paid subscribers

Ghost for membership/premium content

16) ANALYTICS.md

Metrics to Track

page views

impressions

open rate

CTR

read time

scroll depth

shares

replies

subscription conversion

revenue per post

Analytics Loop

Performance data should update:

topic priority

title style

publish time

content length

platform selection

17) MEMORY.md

Memory Categories

best performing topics

best posting times

preferred tone

successful hooks

failed formats

audience reactions

monetized themes

platform behavior

Memory Format Example

MEMORY ENTRY Date: Platform: Topic: Format: Result: Lesson: Action next:

18) LOGS.md

Log Types

publish logs

failure logs

retry logs

API logs

scheduler logs

model routing logs

analytics ingest logs

Log Entry Fields

timestamp

content id

platform

action

status

error message

retry count

response id

19) CHANGELOG.md

v0.1.0

initial blueprint

core structure

agent map

platform matrix

automation design

v0.2.0

API adapter definitions

scheduler rules

memory layer

log framework

v0.3.0

MCP integration layer

analytics loop

monetization logic

fail-safe rules

20) PROMPTS.md

Prompt Families

master system prompt

article prompt

rewrite prompt

SEO prompt

repurpose prompt

scheduler prompt

publish prompt

analytics prompt

memory update prompt

21) PROMPT_MASTER_FINAL.md

MASTER SYSTEM PROMPT

You are the AI Media Orchestrator for a multi-platform publishing ecosystem.

Your job:

turn ideas into high-quality humanic content

repurpose content for each platform

maintain consistency of voice

optimize for clarity, usefulness, and audience trust

prepare publish-ready payloads

respect platform-specific formatting and constraints

support auto schedule, auto post, and daily cadence

update memory from analytics and outcomes

Operating rules:

1. Always produce a canonical master draft first.
2. Rewrite it into platform-specific versions only after the master draft is strong.

3. Make the writing sound human, natural, and direct.
4. Avoid robotic repetition, filler, and generic AI phrases.
5. Keep one source of truth for content and multiple derived variants for distribution.
6. Never publish low-confidence content without quality checks.
7. Always attach title, slug, tags, summary, and CTA where relevant.
8. Log every action.
9. Learn from analytics.
10. Improve future output using memory.

Output format for each content job:

content brief

research notes

master draft

platform variants

SEO pack

schedule suggestion

publish payloads

log entry

memory update

Humanic writing rules:

be direct

be clean

be specific

vary rhythm

avoid over-polished AI tone

keep the voice grounded and useful

Platform adaptation rules:

WordPress: full article, SEO-rich

Medium: clean, readable, minimal friction

Blogger: simple, SEO-friendly

Substack: newsletter tone, personal and sharp

Beehiiv: audience growth and newsletter-native

Mirror.xyz: special-case formatting

Dev.to: technical clarity

Hashnode: developer-first structure

Ghost: premium editorial tone

Auto workflow rules:

if draft confidence is high, schedule immediately

if confidence is medium, route to human review

if source quality is low, do not publish

if platform API fails, retry via queue

if publish succeeds, log and store memory

The system should act like a disciplined media operator, not a noisy content spammer.

22) PROMPT PACK EXAMPLES

Article Prompt

Write one master article from this idea. Make it humanic, structured, and useful. Include title options, summary, and a publish-ready markdown draft.

Repurpose Prompt

Turn this master article into platform-specific versions for WordPress, Medium, Substack, Beehiiv, Dev.to, Hashnode, Ghost, Blogger, and social snippets.

SEO Prompt

Extract the best title, slug, meta description, keywords, and heading structure from this article.

Scheduler Prompt

Given this content and platform list, determine the best publish time and queue order.

Analytics Prompt

Analyze this post performance data and identify what should change in future content.

23) SECURITY.md

Security Rules

keep API keys in secret storage

never hardcode credentials in prompts

separate publish permissions per platform

use least privilege where possible

log auth failures

rotate tokens periodically

24) FAILSAFE.md

Failsafe Rules

queue retries

draft rollback

publish lock if repeated errors

manual override switch

platform rate-limit backoff

content version snapshots

25) OBSERVABILITY.md

Observable Events

draft created

draft edited

scheduled

published

failed

retried

analytics ingested

memory updated

Dashboard Must Show

today's queue

scheduled items

failed items

publish success rate

top platforms

top topics

26) ROADMAP.md

Phase 1 — Foundation

define content schema

define agent roles

build master prompt

build one publishing adapter

Phase 2 — Core Automation

scheduler

repurpose engine

logging

memory updates

Phase 3 — Multi-Platform Expansion

WordPress

Medium

Blogger

Ghost

Dev.to

Hashnode

Phase 4 — Monetization + Analytics

newsletter flow

revenue tracking

performance optimization

Phase 5 — MCP Orchestration

tool routing

modular connectors

deeper automation

27) COSTS.md

Cost Buckets

LLM usage

hosting

database

automation tool

domain

analytics

asset generation

Cost Strategy

route simple tasks to cheaper models

reserve expensive models for final draft

cache outputs

avoid duplicate generation

28) EXAMPLES.md

Example Workflow

1. user drops idea

2. system creates draft

3. editor-agent humanizes it

4. seo-agent packages metadata

5. repurpose-agent builds platform variants

6. scheduler-agent queues the post

7. publisher-agent sends to adapters

8. analytics-agent checks results

9. memory-agent stores learning

29) FINAL IMPLEMENTATION INTENT

Blueprint ini bukan cuma untuk “nulis artikel”. Ini untuk membangun mesin media kecil yang bisa bekerja seperti operator konten semi-otonom.

Arsitektur yang benar:

satu sumber ide

satu master draft

banyak output distribusi

satu loop analytics

satu sistem memory

satu jalur monetisasi

Kalau sistem ini jadi, kamu tidak kerja manual satu-satu lagi. Kamu tinggal mengarahkan, mengedit, dan mengawasi.

30) NEXT BUILD ORDER

Urutan build yang paling waras:

1. finalize schema

2. finalize agents

3. finalize prompt master

4. choose core platform hub

5. build one publisher adapter

6. build scheduler

7. build repurpose engine

8. build log + memory system

9. build analytics loop

10. expand platform by platform

31) DASHBOARD SYSTEM

DASHBOARD.md

PURPOSE

Dashboard menjadi pusat kontrol AI Media Ecosystem.

Fungsi utama:

melihat queue konten

monitoring auto publish

analytics tracking

melihat scheduler

manual override

agent monitoring

API status

platform status

memory inspection

logs inspection

DASHBOARD FEATURES

CONTENT CONTROL

create draft

edit draft

approve publish

force publish

reschedule

archive

PUBLISH STATUS

queued

processing

published

failed

retrying

PLATFORM TRACKING

Track all platforms:

WordPress

Medium

Blogger

Substack

Beehiiv

Ghost

Hashnode

Dev.to

Mirror.xyz

LinkedIn

X

Threads

ANALYTICS PANEL

views

CTR

open rate

subscriber growth

conversion

engagement

best performing topics

best posting time

AI PANEL

model currently used

routing status

token usage

cost usage

generation queue

active agents

MEMORY PANEL

successful hooks

failed content

top niches

audience behavior

trend memory

LOGS PANEL

scheduler logs

API logs

failed publish logs

retry logs

automation logs

32) JSPUTER AI MODE

JSPUTER.md

GOAL

Menjadikan JSputer sebagai lightweight AI operating environment untuk AI Media Ecosystem.

ROLE

JSpoter menjadi:

AI terminal

dashboard host

workflow operator

markdown editor

lightweight IDE

automation console

MCP interface

JSPOTER FEATURES

AI CONSOLE

direct prompt execution

agent invocation

multi-model routing

streaming response

CONTENT STUDIO

markdown editor

article preview

SEO preview

repurpose preview

scheduler preview

WORKFLOW CONTROL

run workflow

stop workflow

retry workflow

debug workflow

PUBLISH CENTER

publish now

queue publish

multi-platform publish

scheduled publish

SYSTEM MONITORING

API health

token usage

scheduler status

queue status

memory status

vector database status

JSPUTER STACK

FRONTEND

Next.js

Tailwind

shadcn/ui

Zustand

Monaco Editor

BACKEND

Node.js

FastAPI optional

Supabase

PostgreSQL

AI

OpenRouter

local LLM optional

MCP server

AUTOMATION

n8n

cron jobs

worker queue

33) PRODUCTION READY ARCHITECTURE

PRODUCTION.md

DEPLOYMENT TARGET

DEV MODE

local docker

local database

sandbox APIs

STAGING

VPS

Cloudflare

isolated queue

PRODUCTION

scalable VPS or cloud

backup database

observability stack

automated retry system

PRODUCTION REQUIREMENTS

REQUIRED

HTTPS

encrypted secrets

queue persistence

rollback support

structured logs

monitoring

rate limiting

caching

OPTIONAL

local LLM

GPU acceleration

vector search

semantic retrieval

AI analytics prediction

34) FULL STACK RECOMMENDATION

FULLSTACK.md

CORE STACK

WRITING

Obsidian

Markdown

AI

OpenRouter

Claude

GPT

Gemini

DeepSeek

DASHBOARD

JSpoter

Next.js

AUTOMATION

n8n

DATABASE

Supabase

PostgreSQL

VECTOR MEMORY

Qdrant

STORAGE

Cloudflare R2

AUTH

Supabase Auth

HOSTING

VPS

Docker

Coolify optional

35) PLATFORM STRATEGY

PLATFORM_STRATEGY.md

CENTRAL HUB

Use WordPress as canonical publishing hub.

Reason:

strongest API

SEO mature

automation friendly

plugin ecosystem

monetization flexible

DISTRIBUTION STRATEGY

Medium

Use for discovery and broad reach.

Blogger

Use for lightweight Google-indexed mirrors.

Substack

Use for email audience ownership.

Beehiiv

Use for newsletter monetization and referral growth.

Dev.to

Use for technical authority.

Hashnode

Use for developer ecosystem reach.

Ghost

Use for premium editorial publishing.

Mirror.xyz

Use for web3-native audience.

36) MASTER BUILD ORDER

BUILD_ORDER.md

PHASE 1 — FOUNDATION

1. repo setup

2. markdown schema

3. database schema

4. prompt system

5. OpenRouter integration

PHASE 2 — AI CORE

1. writer agent

2. editor agent

3. repurpose agent

4. scheduler agent

PHASE 3 — DASHBOARD

1. JSputer integration

2. queue UI

3. logs UI

4. analytics UI

PHASE 4 — PLATFORM ADAPTERS

1. WordPress

2. Dev.to

3. Hashnode

4. Ghost

5. Blogger

6. Medium

7. Substack

8. Beehiiv

9. Mirror.xyz

PHASE 5 — AUTONOMOUS LOOP

1. analytics ingestion

2. memory updates

3. optimization loop

4. AI scheduling

5. AI trend detection

37) FINAL SYSTEM VISION

Sistem ini bukan sekadar autoblog.

Ini adalah:

AI-native media operating system

multi-platform publishing engine

autonomous content workflow

analytics-driven optimization machine

scalable creator infrastructure

Operator hanya:

memberi arah

memberi ide

mengawasi kualitas

mengambil keputusan strategis

Sedangkan sistem menangani:

drafting

repurpose

scheduling

publishing

tracking

optimization

memory

analytics

END OF BLUEPRINT PACK

Blueprint sudah diperluas ke level:

repo-ready

production-ready

AI-native media OS

dashboard architecture

JSpoter integration

multi-platform orchestration

MCP-ready ecosystem

analytics + memory loop

autonomous scheduling & publishing

Sekarang struktur sudah mencakup:

dashboard tracking system

JSpoter AI environment

production deployment architecture

platform strategy

build order roadmap

observability

logging

queue system

memory system

analytics loop

publisher adapters

AI agents

master orchestration prompt

Ini sudah berada di arah yang benar untuk dijadikan:

“AI media operating system” bukan sekadar autoblog biasa.

##

Lanjutan dan tambahan:

38) DATABASE.md

DATABASE PHILOSOPHY

Database bukan cuma penyimpanan data.

Ia adalah:

source of operational truth

orchestration state machine

publish history engine

analytics memory backbone

recovery layer

audit system

Semua workflow harus bisa:

resume

retry

rollback

replay

tanpa kehilangan state.

DATABASE ENGINE RECOMMENDATION

PRIMARY

PostgreSQL

Reason:

relational kuat

JSONB support

full-text search

scalable

mature ecosystem

cocok untuk workflow orchestration

OPTIONAL

SQLite untuk local/dev

Redis untuk queue/cache

Qdrant untuk vector memory

CORE TABLES

users

id

email

username

role

status

created_at

updated_at

workspaces

id

name

owner_id

settings_json
created_at
updated_at

content_items

Canonical master article.

id
workspace_id
title
subtitle
slug
angle
topic
status
master_markdown
summary
source_notes
quality_score
human_review_required
created_at
updated_at
published_at

content_variants

Platform-specific versions.

id
content_id
platform
variant_type
title
body
metadata_json
status
created_at

updated_at

publish_jobs

id
content_variant_id
platform
status
scheduled_time
published_time
retry_count
last_error
response_payload
created_at
updated_at

scheduler_queue

id
job_id
priority
next_attempt
locked_by
lock_until
created_at

analytics_events

id
content_id
platform
metric_type
metric_value
captured_at
raw_payload

memory_entries

id
category
key
value
score
source
created_at

logs

id
service
level
action
message
metadata_json
created_at

api_credentials

id
platform
encrypted_token
refresh_token
expires_at
created_at
updated_at

DATABASE RULES

Rule 1

Master article harus immutable versioned.

Jangan overwrite.

Rule 2

Semua publish attempt wajib dicatat.

Rule 3

Semua scheduler event wajib replayable.

Rule 4

Analytics raw payload wajib disimpan sebelum diproses.

Rule 5

Gunakan soft-delete untuk content.

39) VECTOR_MEMORY.md

PURPOSE

Vector memory dipakai untuk:

gaya tulisan

hook sukses

topik perform

CTA perform

struktur perform

audience preference

semantic recall

trend matching

RECOMMENDED ENGINE

Lightweight

Chroma

Production

Qdrant

Heavy Enterprise

Weaviate

Pinecone

MEMORY TYPES

Writing Memory

Contoh:

intro yang perform tinggi

tone yang paling natural

rhythm tulisan terbaik

Audience Memory

Contoh:

audience suka tutorial pendek

audience tidak suka intro panjang

Platform Memory

Contoh:

Medium suka clean intro

Dev.to suka code-first structure

Monetization Memory

Contoh:

CTA tertentu meningkatkan signup

EMBEDDING STRATEGY

Embed:

article chunks

titles

summaries

hooks

analytics insights

comments

winning CTA

RETRIEVAL STRATEGY

Saat generate artikel baru:

retrieve top related winning content

retrieve top hooks

retrieve tone preference

retrieve related analytics insight

40) OBSERVABILITY.md

PURPOSE

Sistem otomatis tanpa observability = bom waktu.

Harus bisa tahu:

apa yang jalan

apa yang gagal

kenapa gagal

bottleneck dimana

platform mana bermasalah

OBSERVABILITY STACK

Minimum

logs

metrics

alerts

Recommended

Grafana

Prometheus

Loki

IMPORTANT METRICS

AI Metrics

token usage

response latency

failed generations

routing decisions

cost per generation

Publishing Metrics

publish success rate

publish delay

retry count

API failures

Scheduler Metrics

queue depth

stuck jobs

execution time

Analytics Metrics

CTR

read time

conversion

audience retention

ALERT CONDITIONS

Trigger alert when:

publish failures spike

queue stuck

API down

token cost abnormal

analytics ingestion halted

41) FAILSAFE.md

PURPOSE

Sistem autonomous wajib punya rem.

FAILSAFE MODES

SAFE MODE

Semua auto-publish dihentikan.

Draft masih bisa dibuat.

REVIEW MODE

Semua publish wajib human approval.

DRY RUN MODE

Simulasi publish tanpa publish nyata.

FAILSAFE RULES

Rule 1

Jika API error > threshold:

stop auto publish platform itu

Rule 2

Jika quality score rendah:

route ke human review

Rule 3

Jika duplicate detected:

block publish

Rule 4

Jika token budget exceeded:

switch ke cheaper model

Rule 5

Jika queue corruption:

pause scheduler

42) SECURITY.md

SECURITY PHILOSOPHY

Automation tanpa security = liability.

SECURITY LAYERS

Layer 1 — Secret Management

Gunakan:

Vault

Doppler

Supabase secrets

encrypted env

Layer 2 — API Isolation

Pisahkan token:

production

staging

testing

Layer 3 — Permission Scopes

Gunakan least privilege.

Layer 4 — Audit Logs

Semua publish action:

timestamp

actor

payload hash

result

SECURITY RULES

NEVER

hardcode API keys

expose tokens ke frontend

log raw secrets

ALWAYS

encrypt credentials

rotate token

verify webhook signatures

validate payloads

43) DEPLOYMENT.md

DEPLOYMENT STRATEGY

DEV

Laptop

- |— Docker
- |— Local DB
- |— Local Queue
- |— Sandbox APIs

STAGING

VPS

- ├ Next.js
- ├ PostgreSQL
- ├ n8n
- ├ Queue Worker
- └ Monitoring

PRODUCTION

Cloudflare



Load Balancer



App Layer

- ├ Dashboard
- ├ API
- ├ AI Workers
- ├ Scheduler
- └ Publisher Workers

Database Layer

- ├ PostgreSQL
- ├ Redis
- └ Qdrant

Storage Layer

- ├ R2
- └ Backups

DEPLOYMENT RULES

Rule 1

Workers harus stateless.

Rule 2

Scheduler harus persistent.

Rule 3

Queue wajib survive restart.

Rule 4

Backups otomatis harian.

44) MONETIZATION_ENGINE.md

PURPOSE

Bukan cuma publish.

Sistem harus bisa:

grow audience

build owned traffic

convert audience

compound authority

MONETIZATION STACK

Direct

sponsorship

affiliate

premium newsletter

memberships

consulting

digital products

Indirect

authority

leads

hiring magnet

inbound traffic

RECOMMENDED FLOW

Traffic

↓

Email Capture

↓

Newsletter

↓
Trust
↓
Offer
↓
Conversion

CTA STRATEGY

CTA jangan agresif.

Gunakan:

soft CTA

contextual CTA

curiosity CTA

45) TREND_ENGINE.md

PURPOSE

AI harus tahu:

apa yang naik

apa yang jenuh

apa yang punya momentum

TREND SOURCES

Google Trends

Reddit

X

YouTube

Hacker News

RSS feeds

niche newsletters

TREND SIGNALS

velocity

repeated mentions

keyword spikes

sentiment shifts

engagement anomalies

TREND PIPELINE

Signal Detection

↓

Topic Scoring

↓

Priority Queue

↓

Content Suggestion

↓

Publishing Opportunity

46) REVENUE_ANALYTICS.md

TRACK

revenue per article

revenue per subscriber

affiliate conversion

sponsor conversion

best monetized topics

best monetized platforms

IMPORTANT

Kadang artikel dengan traffic kecil lebih profitable.

Jangan cuma optimasi views.

Optimasi:

trust

conversion

retention

47) AI_ROUTING.md

PURPOSE

Tidak semua task perlu model mahal.

ROUTING STRATEGY

Cheap Models

Gunakan untuk:

tagging

formatting

cleanup

metadata

Mid Models

Gunakan untuk:

repurpose

SEO

summaries

Premium Models

Gunakan untuk:

master article

editorial rewrite

strategic content

COST CONTROL

Rules

cache outputs

avoid duplicate prompts

compress context

summarize memory

route intelligently

48) QUEUE_SYSTEM.md

PURPOSE

Queue adalah jantung automation.

JOB TYPES

draft_job

rewrite_job

seo_job

publish_job

analytics_job

retry_job

memory_update_job

QUEUE RULES

Rule 1

Semua job harus idempotent.

Rule 2

Retry exponential backoff.

Rule 3

Dead-letter queue wajib ada.

Rule 4

Job harus resumable.

49) CONTENT_SCORING.md

QUALITY SCORE

Konten harus diberi score sebelum publish.

SCORING FACTORS

Writing

clarity

rhythm

redundancy

grammar

SEO

keyword relevance

heading structure

readability

Humanic

natural phrasing

anti-robotic score

tone consistency

Trust

source confidence

hallucination risk

citation quality

ACTIONS

High Score

Auto schedule.

Medium Score

Human review.

Low Score

Reject or rewrite.

50) FINAL VISION — TRUE ENDGAME

Tujuan akhirnya bukan sekadar “AI buat artikel”.

Endgame sebenarnya:

autonomous media infrastructure

scalable authority engine

AI-assisted publishing company

audience ownership machine

compounding knowledge system

Sistem ini nanti bisa berkembang menjadi:

niche media network

AI newsroom

research publishing engine

educational media ecosystem

automated documentation company

creator operating system

AI-native publishing startup

Dan kalau arsitekturnya benar dari awal:

scale jadi jauh lebih mudah

automation tidak chaos

quality tetap terjaga

monetization lebih sehat

operator tidak burnout